Graphs

- Graph is a data structure, similar to trees. We can say that trees and Linked lists are special cases of graph.

- Graph contains nodes (or) Vertexes which are connected using edges. where every node contains some information similar to Linked lists and trees.

- We can model a lot of real world situations using graphs and solve real problems by solving the underlying graph problem. Just find a way to represent real world objects using Vertices and the relations b/w real world objects as edges between the graph

- Edges can be directed (or) undirected

directed:
if (u,v) is directed
you can say v is neighbour of u
but u is not neighbour of v.

undirected:
if (u,v) is undirected
you can say u is neighbour of v
and v is neighbour of u.

Real world examples that you can model as graphs,

① Facebook friends → facebook accounts as Vertices
Friend relations as edges.
↓
Undirected, unweighted
↓
because friendship is like in this case
if A is friend of B,
B is friend of A.

unweighted because all friend relations are same,
they are not representing any quantity

(2) Instagram follow relations → directed, unweighted graph b/w user accounts

bcoz follow needs direction who is following who.

→ Graph of all currently pending friend requests can be seen using directed, unweighted graph.

→ no one uses graph, just an analogy.

(3) Road networks b/w Cities — undirected, weighted. graph where vertices/Nodes represent cities.

Edges represent roads b/w cities.

Weights on edges represent distance/time to cover the road.

And we can also solve real world Problems using equivalent graph problems. examples:-

(1) Shortest path finding b/w 2 nodes in a graph (weighted, directed/undirected)

Application: (1) Fastest driving route in google maps

(2) Fastest Path to route a network packet

(3) robots moving inside a warehouse

(2) Minimum Spanning tree → a subset of edges in a graph sufficient enough to connect all nodes.

↓

the Spanning tree with the least amount of weight

Applications : ① given a set of locations on a map and distances between all of them. finding cheapest way to lay internet cables to connect all location with least amount of wire.

③ Topological sort Problem

Applications : ① Dependency resolving in installing Software Packages

② Finding out order in which we have to register for courses so that all prerequisite constraints will be satisfied.

and so on.

In this course we will see limited problems, you may study more graph problems in any Algorithms Course in future.

We will look at in this module

DFS, BFS → graph traversal Algorithms

Dijkstra, Floyd-Warshall Algorithms for Shortest Path finding Problem

Representation of graphs (in a program)

2 main ways

1. Adjacency matrix
2. Adjacency List

Adjacency matrix :-

Vertices as an array V

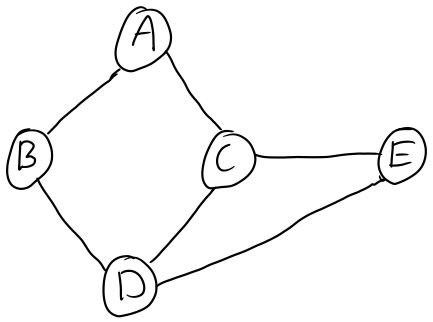
Edges are stored in a matrix $E_{v \times v}$.

where $e_{uv} = 1$ if (u, v) is an edge

$= 0$ otherwise

if Graph is weighted, you can store weights in e_{uv} and make e_{uv} as some special character '_' if u,v is not an edge.

ex:-



index 0 1 2 3 4

$V = [A, B, C, D, E]$

$$E = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{ccccc} 0 & 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

Note :-

(i) if our graph is undirected then always $e_{uv} = e_{vu}$

So for undirected graphs $E^T = E$

↳ Transpose of E

for directed graphs, we can not say the same.

(ii) if $|V| = n$ then $|E| = |V|^2$

↓ no. of vertices, Size of vertex set

↓ Size of matrix E.

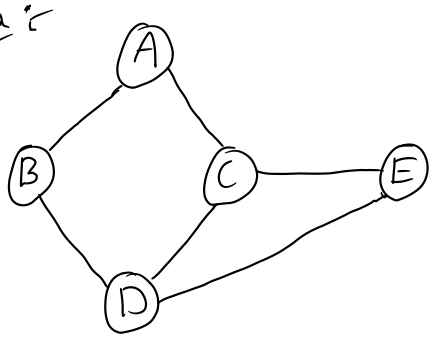
Adjacency list :-

Vertexes are stored in an array V.

Edges are stored in an array of array.

For every vertex $u \in V$, we have an array of all neighbours of u and our adjacency list is simply array of all such arrays for every vertex.

eg :-



index 0 1 2 3 4

V = [A, B, C, D, E]

adjacency list,

A → [B, C]

B → [A, D]

C → [A, D, E]

D → [B, C, E]

E → [C, D]

Size of adjacency list = { 2 × no. of edges (for undirected graph)
no. of edges (for directed graph)

when to use adjacency matrix and adjacency list ?

→ for n nodes in a graph, min. no. of edges = 0

max. no of edges = $^nC_2 = \frac{n(n-1)}{2} = O(n^2)$

if $|E|$ is close to $O(n^2)$ → call it a dense graph → use adjacency matrix

$|E|$ = no. of edges

if $|E|$ is close to $O(n)$ → call it a sparse graph → use adjacency list

Now we will first look at DFS and then BFS

DFS → Depth First Search

BFS → Breadth first search.

They are 2 main simple Algorithms/approaches to travel through our graph and visit every node.

DFS → main idea: Call explore on some node u.

u will visit all nodes in the graph reachable from u in depth first fashion

Algorithm :-

Keep calling explore on neighbour vertices and only backtrack when all current neighbours are explored.

```
public int time = 0;
public int visited[];
public int previsit[];
public int postvisit[];

public  void explore(int[][] E,int n){
    if(visited[n]==1)return ;
    visited[n] = 1;
    time++; previsit[n]=time;
    for(int i=0;i<E[n].length;i++){
        if(visited[E[n][i]] == 0){
            explore(E,E[n][i]);
        }
    }
    time++; postvisit[n]=time;
}

public  void DFS(int[][] E){
    int V = E.length;
    visited = new int[V];
    previsit = new int[V];
    postvisit = new int[V];
    for(int i=1;i<V;i++){
        if(visited[i]==0)explore(E,i);
    }
}
```

→ DFS implementation

```
public void main(String[] args){
    int [][] E= {
        {},{},{1,3},{1,4},
        {2},{6},{1,3},{},
        {7,9},{}
    // E is your adjacency list
    // of edges indexed by
    // vertex numbers
    };
    DFS(E);
}
```

↓ main function

output (after adding print statements in DFS) ↓

```
C:\Users\K REVANTH\Desktop\java_practice>javac graphs.java

C:\Users\K REVANTH\Desktop\java_practice>java G
previsit times : 1 3 4 5 9 10 13 15 16
postvisit times : 2 8 7 6 12 11 14 18 17
```

DFS explanation

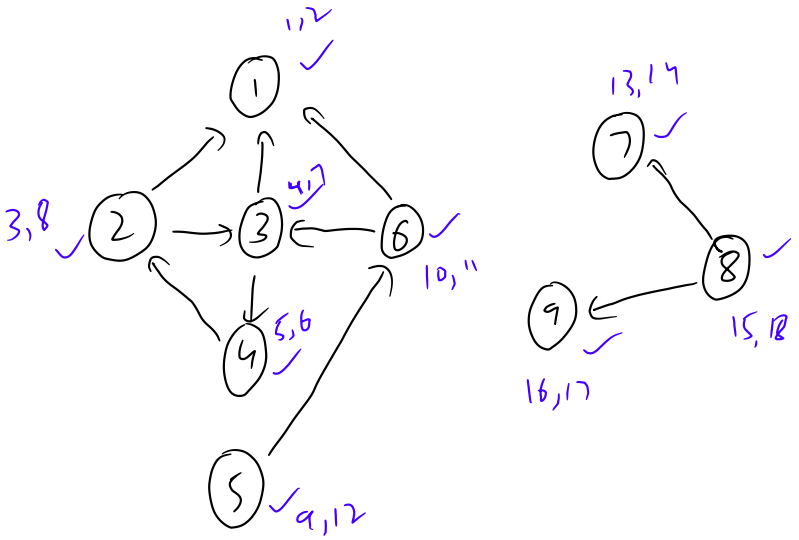
Explore (n) where 'n' is some node explores all the nodes reachable from node n in depth first fashion.

→ For all nodes we visit, we mark visited[n] = 1 and we will record the previsit[n] as the time when we are visiting a node

→ At end of explore(n), when all neighbours of n are done exploring. It will update postvisit[n] as the current time and back tracks.

→ DFS calls explore on all unvisited nodes in order.

In the code we saw in previous pase, we are doing for the following example.



initially time = 0,

all visited array is 0

So DFS calls explore on '1'

→ only '1' is explored since '1' has no outgoing edges.

→ then call explore on 2 which explores 2,3,4

→ 5 → explores 5,6

→ 7 → only 7

→ 8 → 8,9

and keep track of pre, post visit times.

BFS Breadth First Search

— Depth first search when started at a particular node goes to its unexplored neighbour and then to its neighbour, ---so on only backtracks when all current neighbours are explored and then goes back to parent, Parent again goes deep to its other unexplored neighbour .... so on.

— DFS goes as deep as possible every time.

→ BFS on contrast first visits all current neighbours (basically all nodes which are at a distance of single edge from our starting node)

→ And then looks at all neighbours of current neighbours. (all nodes at distance 2) and distance 3, so on ....

— There is no post visit in BFS.

Algorithm :-

```
public static void BFS(int[][] E, int n){
    Queue q = new Queue(E.length);
    q.push(n);
    int visited[] = new int[E.length];
    visited[n] = 1;
    while(q.isEmpty() == false){
        int x=q.peek();
        System.out.print(x + " ");
        q.pop();

        for(int i=0;i<E[x].length;i++){
            if(visited[E[x][i]] == 0){
                q.push(E[x][i]);
                visited[E[x][i]]=1;
            }
        }
    }System.out.print("\n");
}
```

BFS function called on a particular node

```
public void main(String[] args){
    int [][] E= {
        {},{2,3,6},{1,3,4},{1,2,4,6},
        {2,3},{6},{1,3,5},{8},
        {7,9},{8}
    // E is your adjacency list
    // of edges indexed by
    // vertex numbers
    };
    BFS(E,1);
}
```

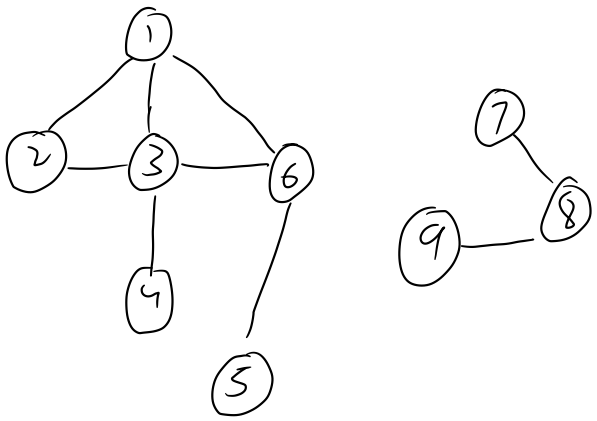
main function

output

```
C:\Users\K REVANTH\Desktop\java_practice>javac graphs.java

C:\Users\K REVANTH\Desktop\java_practice>java G
1 2 3 6 4 5
```

We ran BFS on the following graph,



(undirected version of previous example)

→ When we call BFS on only node 1 it only explores nodes reachable from 1.

First 1, 2, 3, 6, 4, 5

distance from '1' 0 1 2

First covers nodes at distance 1, then distance 2, 3, ----..

Implementation

DFS implementation is simple, just use recursion, use a visited array.

→ For BFS you need to use queues.

idea for BFS: (you should've come to class for better explanation)

→ Start with empty queue, insert starting node.

→ Now in a loop, until queue becomes empty, Pop front element in queue and push all unvisited neighbours of it in back of queue.

→ Do this until queue becomes empty.

Note:

If you follow above method, at any point of time your queue will only contain nodes at distance 'n' from (1) and distance (n+1) from (1)

BFS, DFS are just traversal Algorithms in graphs, you can travel any way you want.

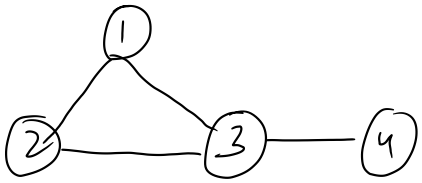
↓ similar to preorder, inorder, Postorder in trees.

Cycles in graphs

what is a cycle in a directed graph?

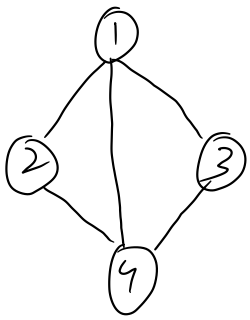
Ex:

(i)



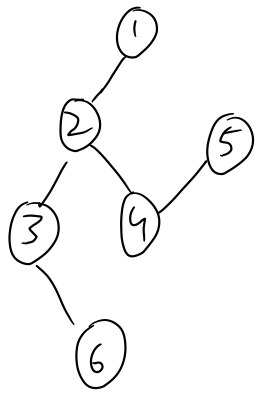
1, 2, 3 is a cycle

(ii)



(1, 3, 4),
(1, 2, 4),
(1, 2, 4, 3)
are all cycles

(iii)

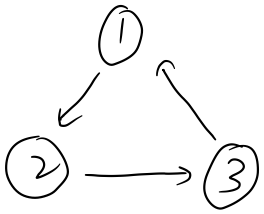


No cycles here

what is a cycle in an undirected graph?

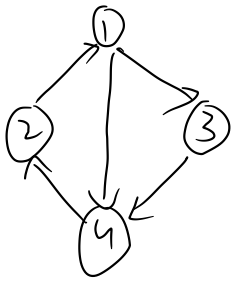
ex:-

(i)



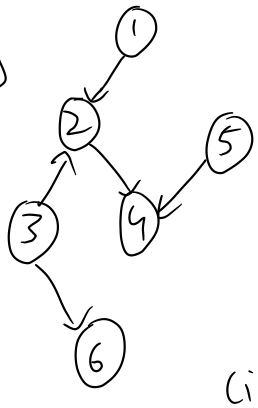
(1, 2, 3) is a cycle

(ii)



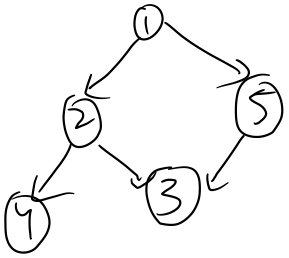
(1, 4, 2)
(1, 3, 4, 2)
are cycles
(1, 3, 4) is not cycle

(iii)



No cycles here

(iv)



no cycles here also.

If your graph contains a cycle → Call it **Cyclic graph**

if not → Call it **acyclic graph**

When we model real situations using graphs, like transport networks (or) Internet network (or) dependencs graphs in Software (or) any other situations,

There exist some situations where cycle is a Problem

ex: (1) Course Prerequisite Problem

↓

Courses list → every Course as a vertex

Prerequisite relation b/w 2 Courses → directed edge.

Here we cannot have cycles.

(2) Deadlock in operating systems.

and more.

These kind of situations where containing a cycle is a Problem.

In next Page we will see Cycle detection problem.

Cycle detection problem

Given a graph (V, E) output whether it contains a cycle.

directed (or) undirected graphs.

Algorithm

(1) Apply DFS we studied before and findout previsit, Postvisit values of every node.

(2) For all edges in your graph $(u, v) \in E$ where $u, v \in V$

Do the following 3 checks

(i) $pre(u) < pre(v) < post(v) < post(u)$ } → tree edge

(ii) $pre(u) < post(u) < pre(v) < post(v)$

(or)

$pre(v) < post(v) < pre(u) < post(u)$

} → cross edge

(iii) $pre(v) < pre(u) < post(u) < post(v)$ } → back edge

After doing DFS, calculating pre visit, Post visit of all nodes

Do this check on all graph edges.

→ Every edge will only fall into one of the 3 categories

→ if there exist atleast one backedge ⟹ Cyclic graph

Directed, undirected graph. Just give it in adjacency list format it will work.

In the given textbook, read the following pages

91 - (100) → till above Section 3.3.2

(and)

115 - 118

→ read rest of ch3 too if you want. good book. currently not in syllabus. Maybe in next year algorithms course you'll study more.